Operating Systems
Week 4
Lecture/Discussion 4

Junaid Abdul Wahid

Previous

- Concurrency: Mutual Exclusion and Synchronization

Today Discussion

- Monitors
- Synchronization
- Process Interaction
- Readers/Writers Problem

Monitors

The monitor is a programming-language construct that provides equivalent functionality to that of semaphores and that is easier to control.

Implemented in a number of programming languages, including

Concurrent Pascal, Pascal-Plus,
Modula-2, Modula-3, and Java.

Chief characteristics

Local data variables are accessible only by the monitor
Process enters monitor by invoking one of its procedures
Only one process may be executing in the monitor at a time

Synchronization

Synchronisation achieved by **condition variables** within a monitor

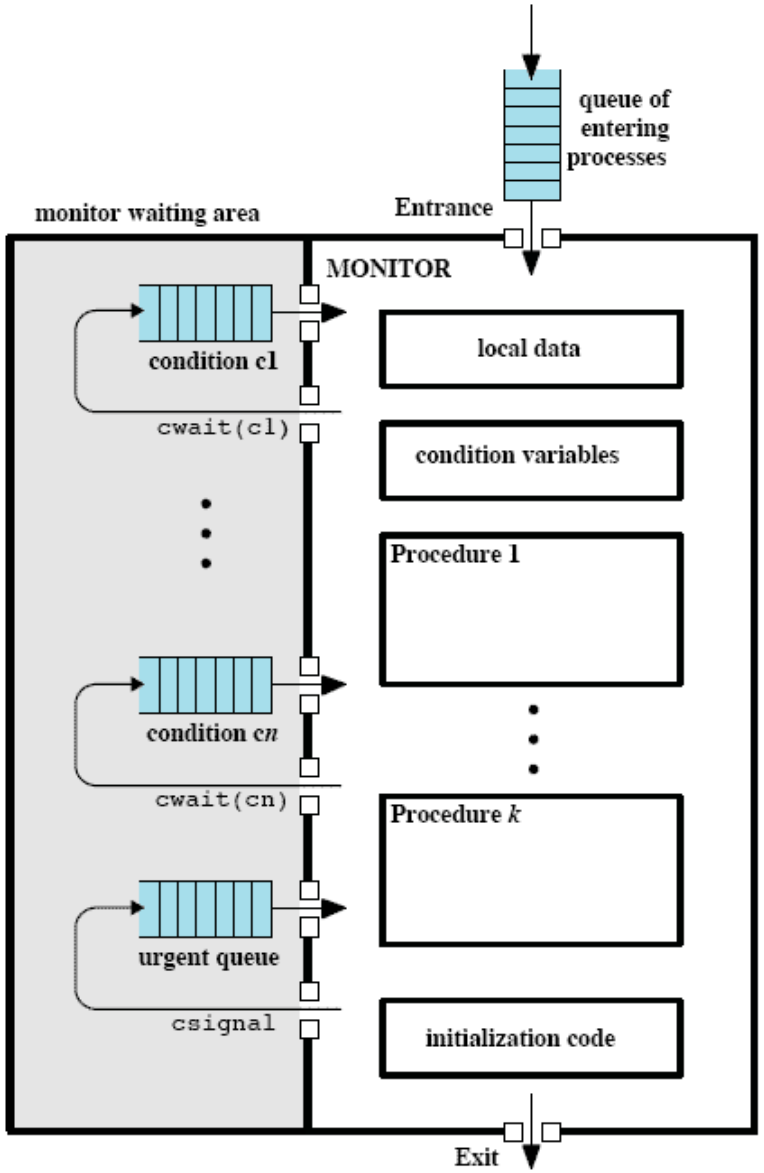
only accessible by the monitor.

Monitor Functions:

Cwait(c): Suspend execution of the calling process on condition *c*

Csignal(c) Resume execution of some process blocked after a cwait on the same condition

Structure of a Monitor



Roadmap

- Principals of Concurrency
- Mutual Exclusion: Hardware Support
- Semaphores
- Monitors
- Message Passing
- Readers/Writers Problem

Process Interaction

When processes interact with one another, two fundamental requirements must be satisfied:

- synchronization and
- communication.

Message Passing is one solution to the second requirement

- Added bonus: It works with shared memory *and* with distributed systems

Message Passing

The actual function of message passing is normally provided in the form of a pair of primitives:

send (destination, message)
receive (source, message)

Synchronization

Communication requires synchronization
- Sender must send before receiver can receive

What happens to a process after it issues a send or receive primitive?
- Sender and receiver may or may not be blocking (waiting for message)

Blocking send,
Blocking receive

Both sender and receiver are blocked until message is delivered
Known as a *rendezvous*
Allows for tight synchronization between processes.

Non-blocking Send

More natural for many concurrent programming tasks.

Nonblocking send, blocking receive

- Sender continues on
- Receiver is blocked until the requested message arrives

Nonblocking send, nonblocking receive

- Neither party is required to wait

Addressing

Sending process need to be able to specify which process should receive the message

- Direct addressing
- Indirect Addressing

Direct Addressing

Send primitive includes a specific identifier of the destination process
Receive primitive could know ahead of time which process a message is expected
Receive primitive could use source parameter to return a value when the receive operation has been performed

Indirect addressing

Messages are sent to a shared data structure consisting of queues
Queues are called *mailboxes*
One process sends a message to the mailbox and the other process picks up the message from the mailbox

Indirect Process Communication

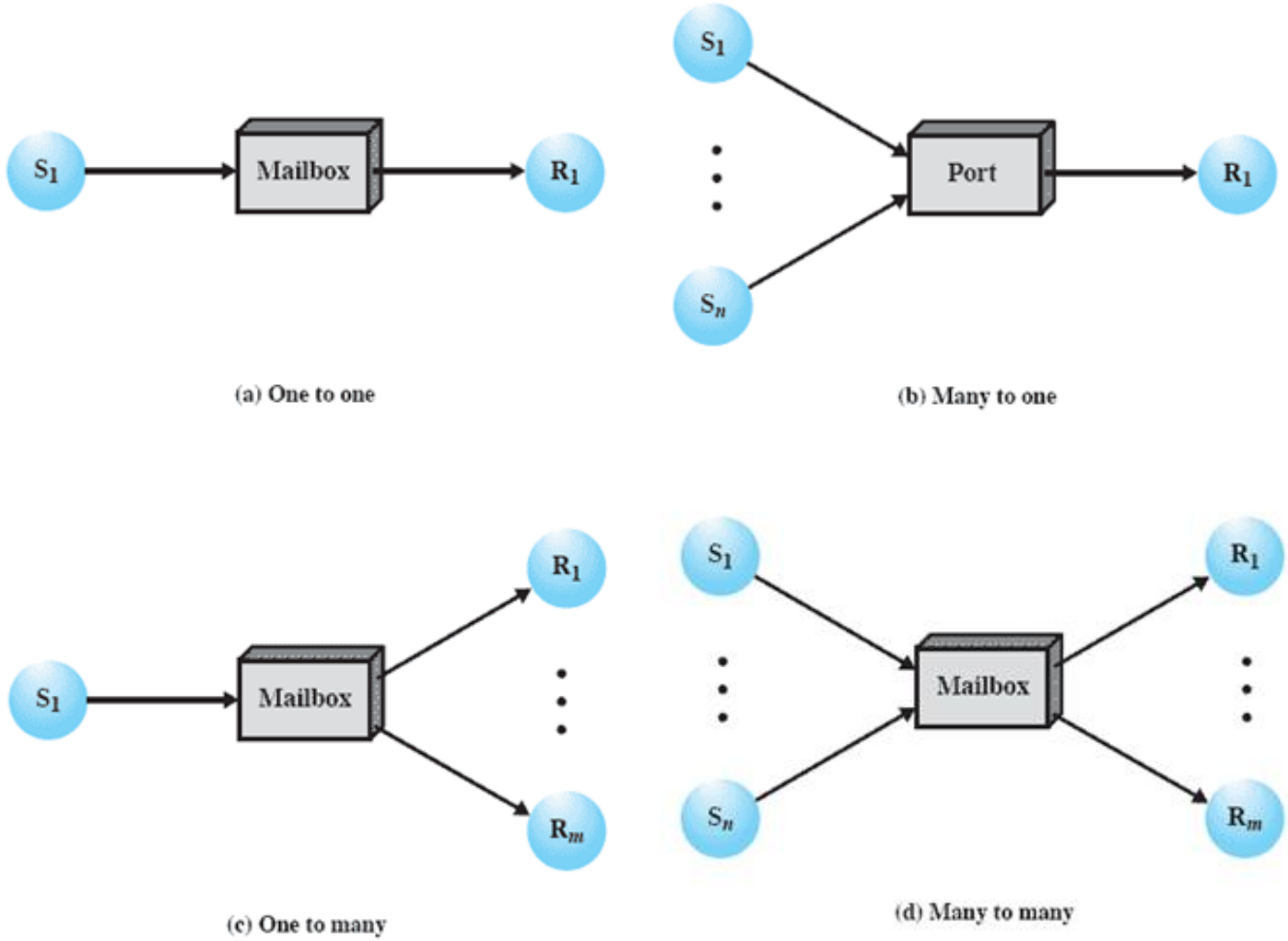


Figure 5.18 Indirect Process Communication

General Message Format

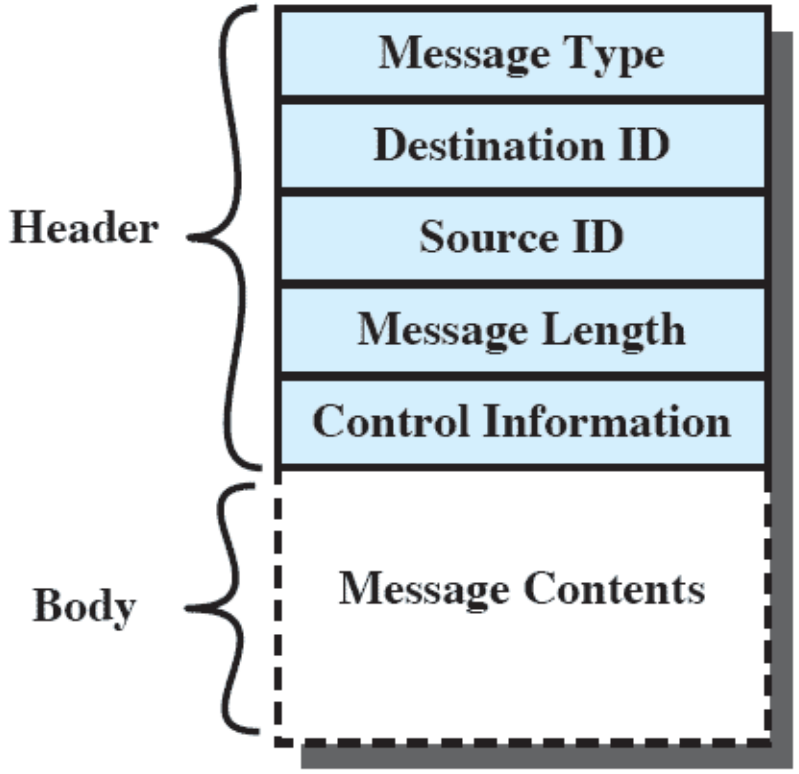


Figure 5.19 General Message Format

Readers/Writers Problem

A data area is shared among many processes

- Some processes only read the data area, some only write to the area

Conditions to satisfy:

1. Multiple readers may read the file at once.
2. Only one writer at a time may write
3. If a writer is writing to the file, no reader may read it.

interaction of readers and writers.